Minimax a Alpha-Beta

En la llamada inicial:
alfa es el menor número posible: -FLT_MAX
beta el mayor número posible: FLT_MAX

```
function Minimax(node,depth,maximPlayer)
   if (depth == 0) or node ∈ TerminalNodes
      return h(node)
   if maximPlayer == TRUE
      bestValue ⟵ −∞
      for each child of node
         v ⟵ Minimax(child,depth-1,FALSE)
         bestValue ⟵ max(bestValue,v)
      return bestValue
   else          // minimizing player
      bestValue ⟵ +∞
      for each child of node
         v ⟵ Minimax(child,depth-1,TRUE)
         bestValue ⟵ min(bestValue,v)
      return bestValue
```

```
function AlphBe(node,depth,maximPlayer,α◁,β◁)
   if (depth == 0) or node ∈ TerminalNodes
      return h(node)
   if maximPlayer == TRUE
      bestValue ⟵ −∞
      for each child of node
         v ⟵ AlphBe(child,depth-1,FALSE,α,β)
         bestValue ⟵ max(bestValue,v)
         α ⟵ max(α,v)                          ◁
         if (β≤α)                              ◁
            return bestValue                   ◁
      return bestValue
   else          // minimizing player
      bestValue ⟵ +∞
      for each child of node
         v ⟵ AlphBe(child,depth-1,TRUE,α,β)
         bestValue ⟵ min(bestValue,v)
         β ⟵ min(β,v)                          ◁
         if (β≤α)                              ◁
            return bestValue                   ◁
      return bestValue
```

Convertir minimax en alphabeta (I)

- Copiar el archivo minimax.h a alphabeta.h y modificar:
  - _MINIMAX_H por _ALPHABETA_H
  - La declaración de la función minimax() para que sea alphabeta() con sus nuevos argumentos.
- Copiar el archivo minimax.cpp a alphabeta.cpp y modificar:
  - #include <minimax.h> por #include <alphabeta.h>
  - La declaración de la función minimax() por la de alphabeta().
  - Todas las llamadas minimax() por llamadas a alphabeta().
  - Hacer los cambios en el algoritmo para que sea alphabeta.

Convertir minimax en alphabeta (II)

- Modificar el archivo otello_test.cpp:
  - #include <minimax.h> por #include <alphabeta.h>
  - Las llamadas a minimax() se cambian por llamadas a alphabeta().

Inserción de un contador de llamadas (I)

- En el archivo heuristic.cpp añadir una variable global, una función y modificar la función h():

```
static unsigned int mi_contador_de_llamadas=0;

unsigned int GetCalls()

{

 return mi_contador_de_llamadas;

}

float h(Player pl,Board b)

{

 mi_contador_de_llamadas++;

…
```

Inserción de un contador de llamadas (II)

- En el archivo heuristic.h añadir la declaración de la función, por ejemplo:

```
…
unsigned int GetCalls(void);
float h(Player pl,Board b);
```

Inserción de un contador de llamadas (III)

- En el archivo otello_test.cpp añadir la cabecera heuristic.h y la línea para que muestre las llamadas:

```
#include "minimax.h"
#include "heuristic.h"
…
if (DEB)
{
 cout << "Las llamadas son: " << GetCalls() << endl;
 cout << "The winner is " << Name(winner) << …
…
```

Varias heurísticas (I)

- En el archivo heuristic.cpp crear las nuevas funciones con las heurísticas. OJO: no borrar la función h() ya existente.

```
float h1(Player pl,Board b){ // Mi heurística 1
…
return valor;
}
float h2(Player pl,Board b){ // Mi heurística 2
…
return val;
}
```

Varias heurísticas (II)

- En el archivo heuristic.h añadir líneas para cada nueva heurística:

```
float h(Player pl,Board b); // Heurística existente, no borrar
float h1(Player pl,Board b); // Mi heurística 1
float h2(Player pl,Board b); // Mi heurística 2
```

Varias heurísticas (III)

- En el archivo alphabeta.cpp modificar el código de llamada a la heurística:

```
if ( depth==0 || sons.empty() )
{
 // float ret=h(pl,b,false); // Comentado
 float ret=(pl==Alice) ? h1(pl,b) : h2(pl,b); // Añadido
 if (DEB && (depth==chosen_depth))
  cout << "No movements from the initial depth. Returning " << ret << endl;
 return ret;
}
```